Akademia Górniczo-Hutnicza im. Stanisława Staszica w Krakowie
| Wydział Inżynierii Mechanicznej i Robotyki | Katedra Robotyki i Mechatroniki

INŻYNIERIA MECHATRONICZNA | I stopień | semestr VI | 2025/2026

Programowanie Obiektowe i Inżynieria Oprogramowania

Instrukcja C++/CLI:
6. Architektura aplikacji

Wprowadzenie do aplikacji okienkowych w C++

Nauczysz się:

- tworzyć autorską bibliotekę statyczną oraz dołączać ją do projektu,
- dzielić program na projekty w ramach Rozwiązania Visual Studio,
- koncepcji stosowania i implementacji wrapperów.

Dodatkowe materiały:

- Materiały do zajęć: [link],
- oficjalna strona C++/CLI [link].

Kierunkowe efekty kształcenia pokryte w tej instrukcji:
IME1A_U1, IME1A_U4

Koordynator przedmiotu:
dr inż. Lucjan Miękina, miekina@agh.edu.pl

Autor instrukcji:
dr inż. Jakub Bryła, jakub.bryla@agh.edu.pl

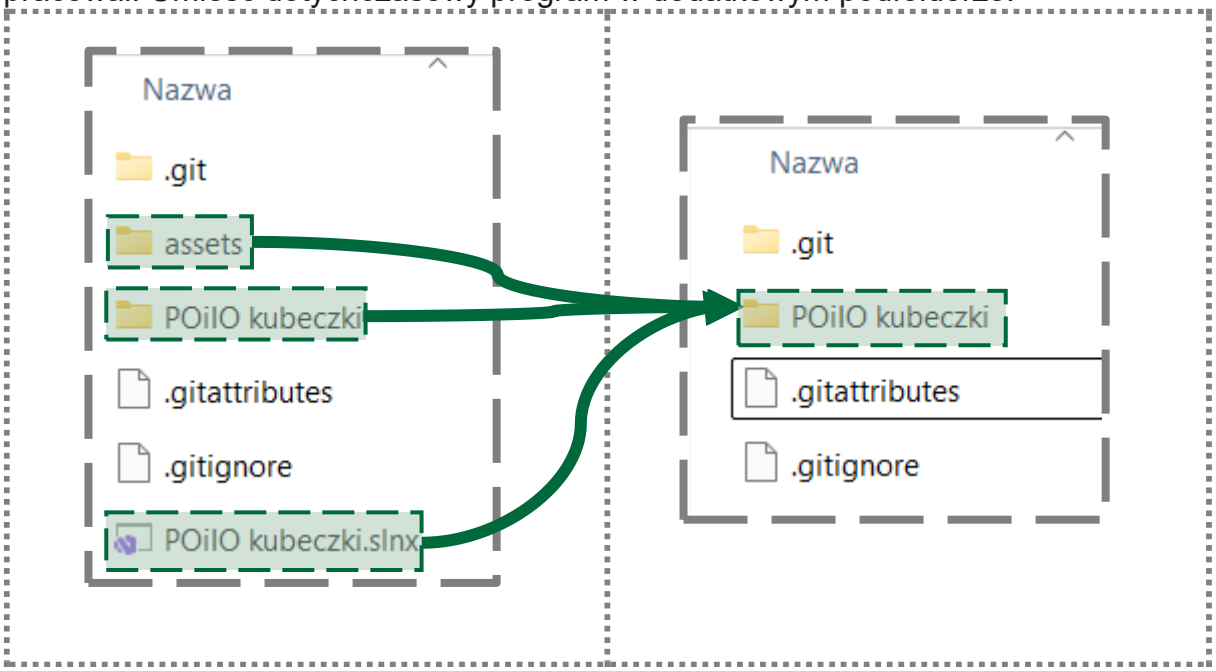
1. Wprowadzenie

Przebudujemy projekt kubeczków, tak aby podzielić kod na logiczne moduły. Głównym elementem projektu będzie biblioteka statyczna, zawierająca *backend* aplikacji. Następnie, w ramach Rozwiązania VisualStudio, dodamy kolejny projekt, do którego załączymy autorską bibliotekę i zaimplementujemy kod odpowiedzialny za wyświetlanie informacji w konsoli.

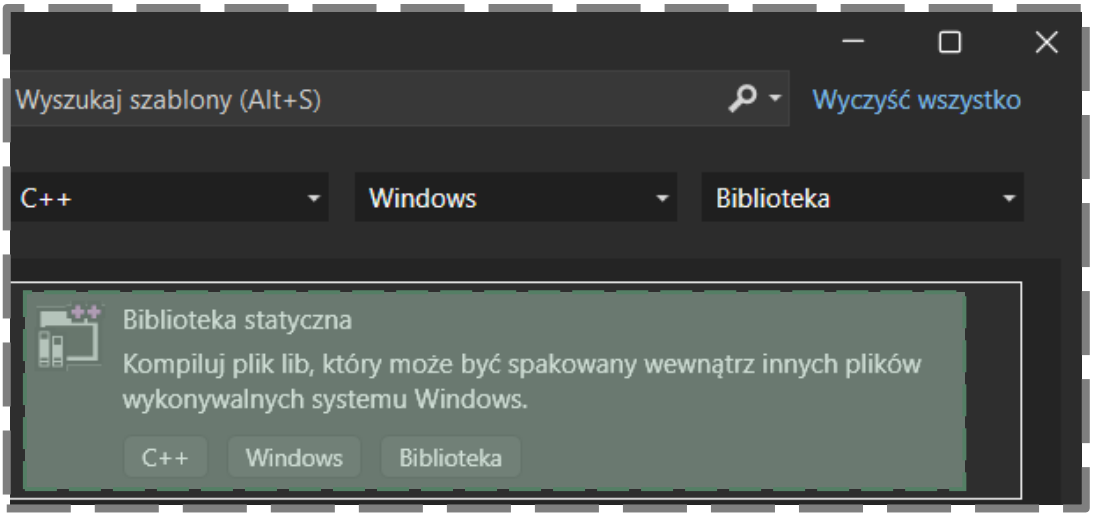
2. Zadania

ZADANIE 2.1 – Utworzenie nowego projektu, backend jako biblioteka statyczna

Sklonuj repozytorium zdalne. Następnie utwórz nową gałąź, na której będziesz pracował. Umieść dotychczasowy program w dodatkowym podfolderze:

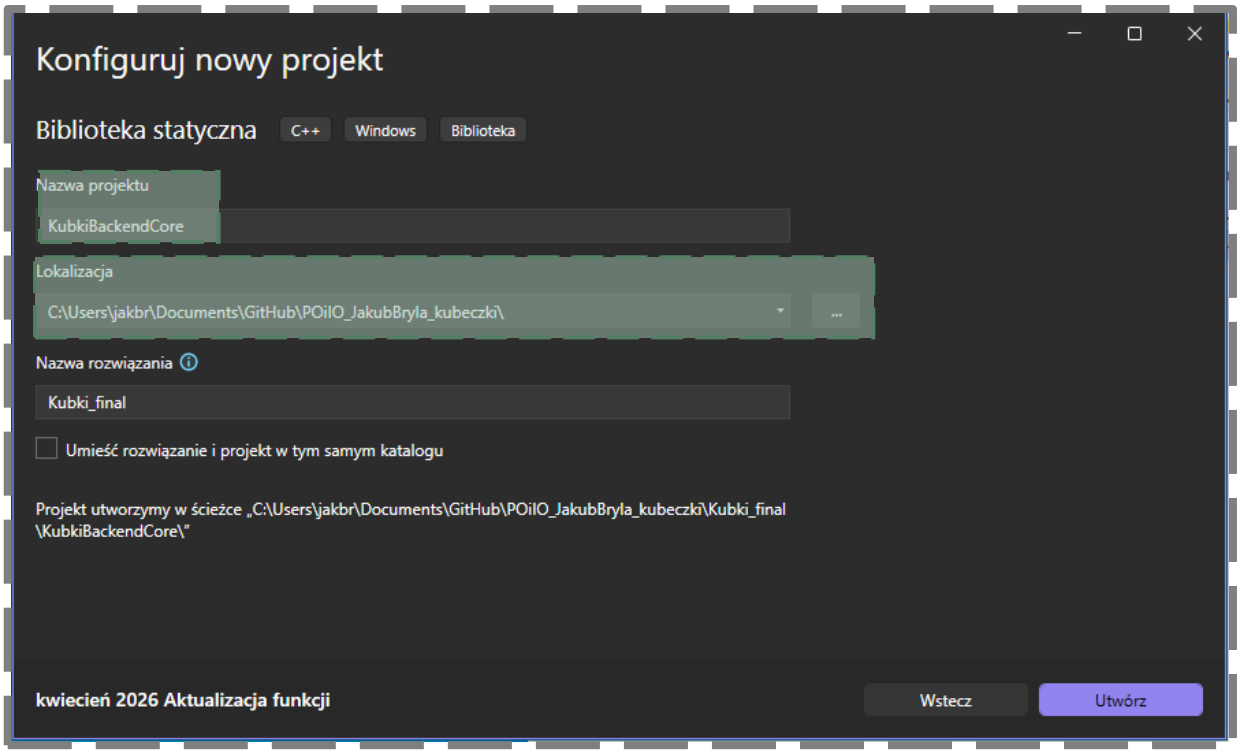


Następnie, utwórz nowy projekt wybierając szablon „Biblioteka statyczna” [ang. *Static Library*]:

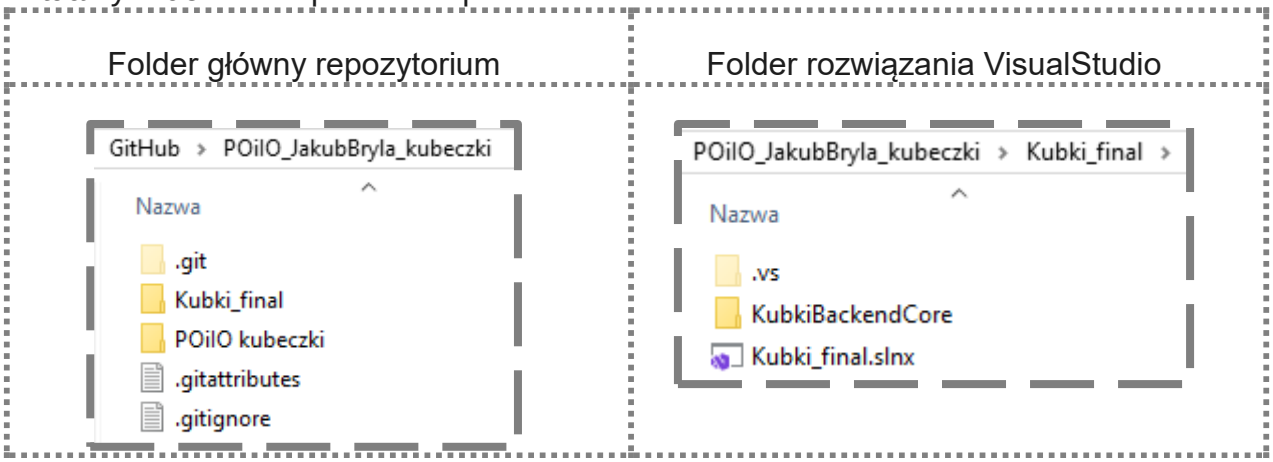


W kolejnym oknie podaj:

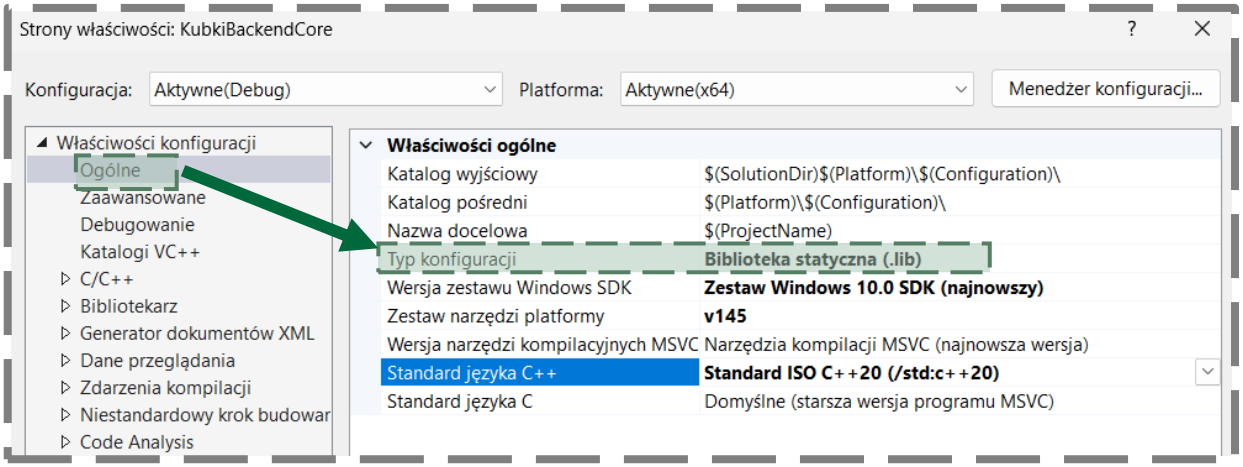
- nazwę projektu: „KubkiBackendCore”,
- lokalizację projektu – główny folder repozytorium lokalnego,
- nazwę rozwiązania: „Kubki_final”.

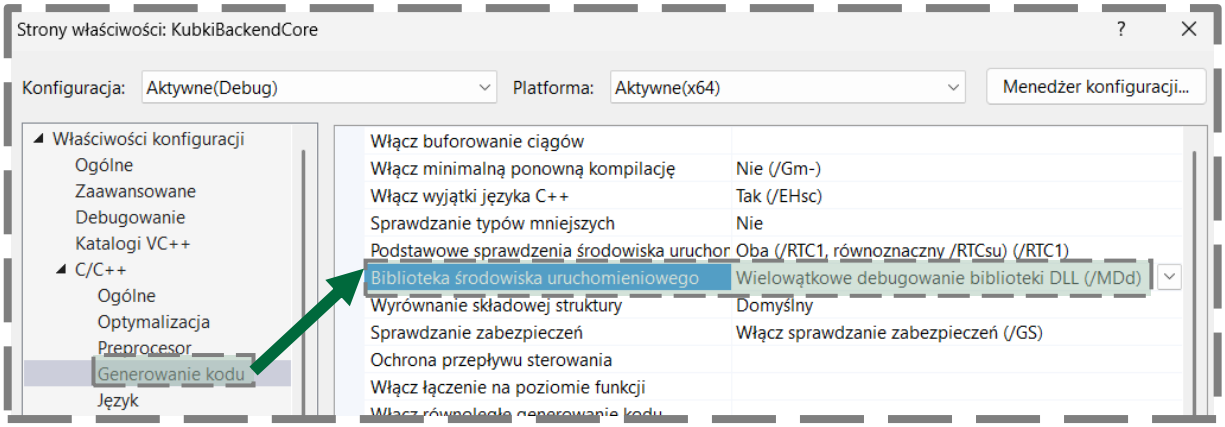


Aktualny widok w Eksploratorze plików:

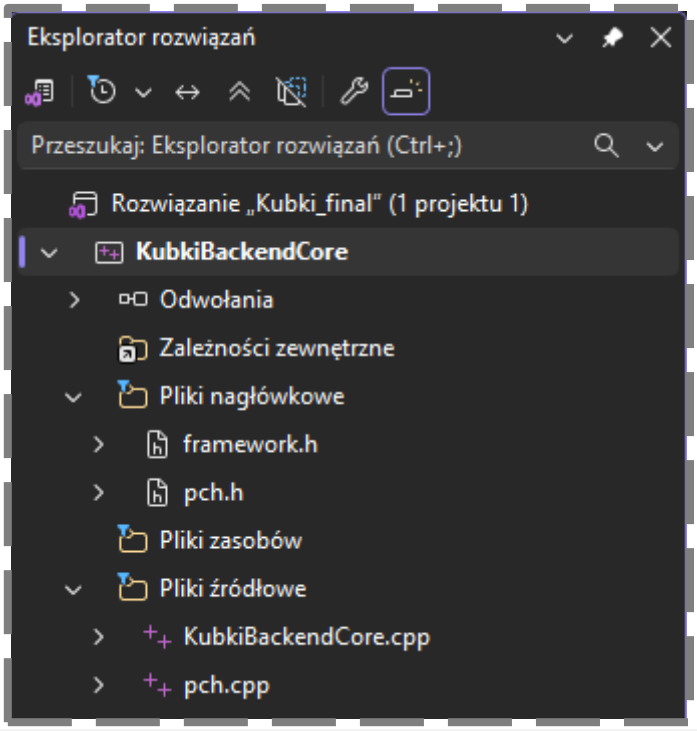


Upewnij się, że wszystkie ustawienia projektu są prawidłowe. W drzewku projektu kliknij prawy przycisk myszy (PPM) na nazwę projektu i wybierz „Właściwości”:

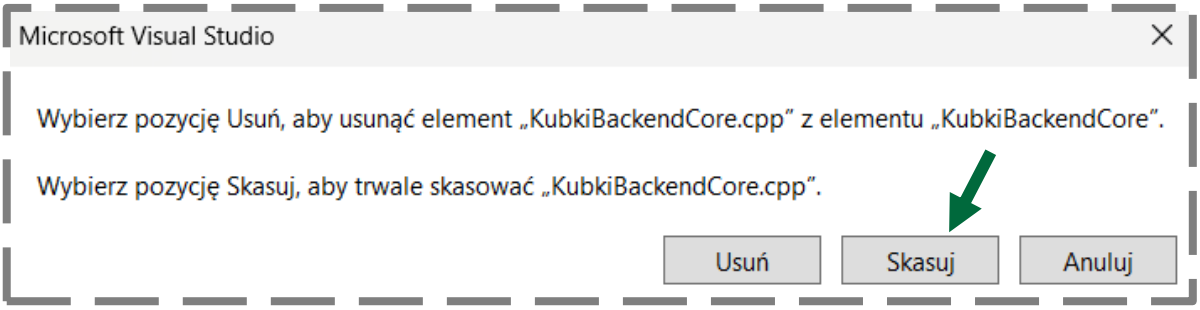




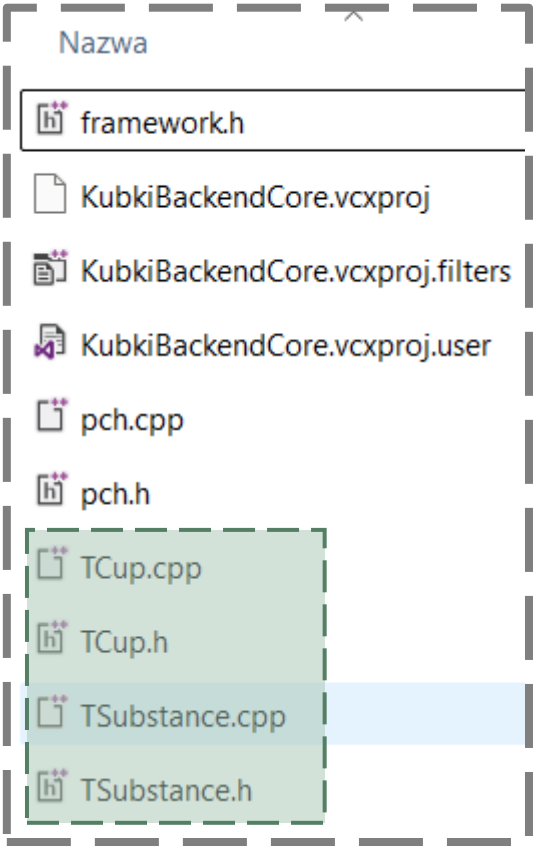
Widok drzewka projektu:



Skasuj plik z rozszerzeniem „.cpp” o nazwie zgodnej z nazwą projektu – w moim przypadku „KubkiBackendCore.cpp”:

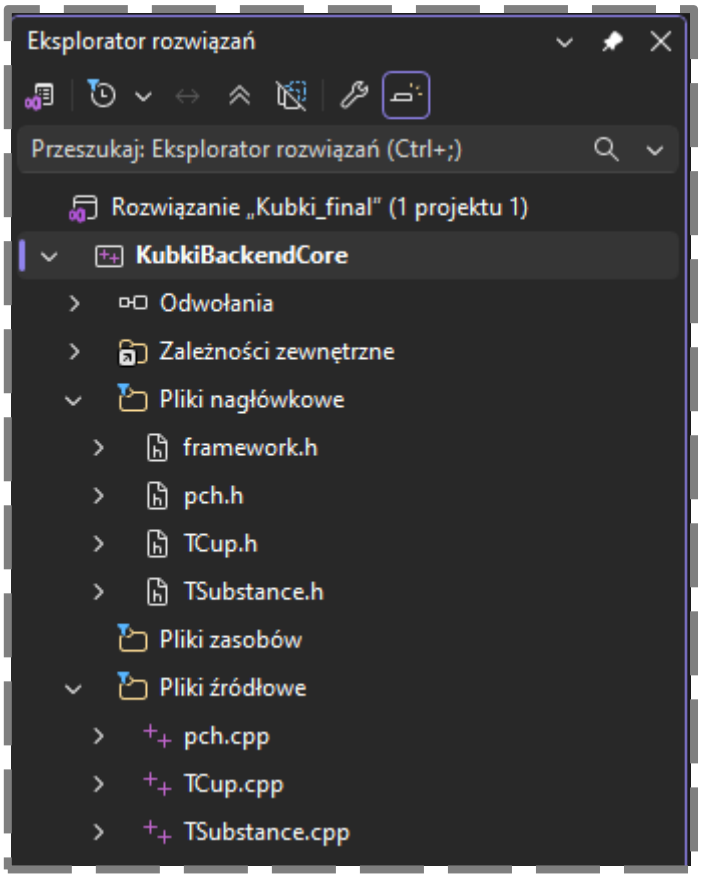


Do folderu projektu skopiuj, z poprzedniej wersji programu, pliki nagłówkowe i źródłowe klas *TSubstance* i *TCup*:

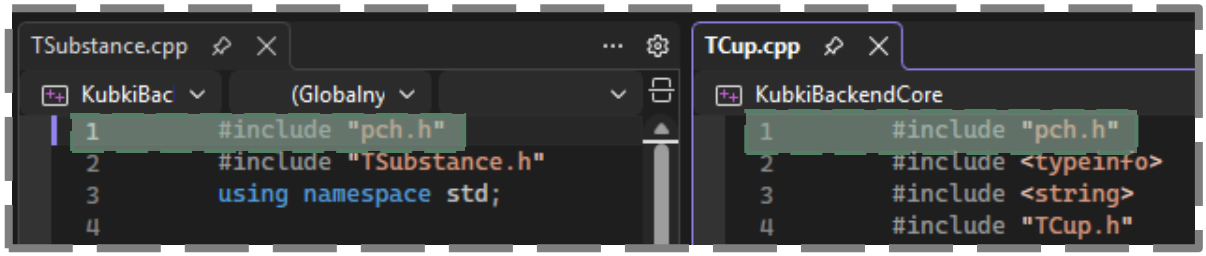


W drzewku projektu kliknij PPM odpowiednio na folder „Pliki nagłówkowe” i „Pliki źródłowe”, wybierz opcję „*Dodaj*” → „*Istniejący element…*” i wybierz dodane do folderu pliki, odpowiednio z rozszerzeniem *.h* i *.cpp*.

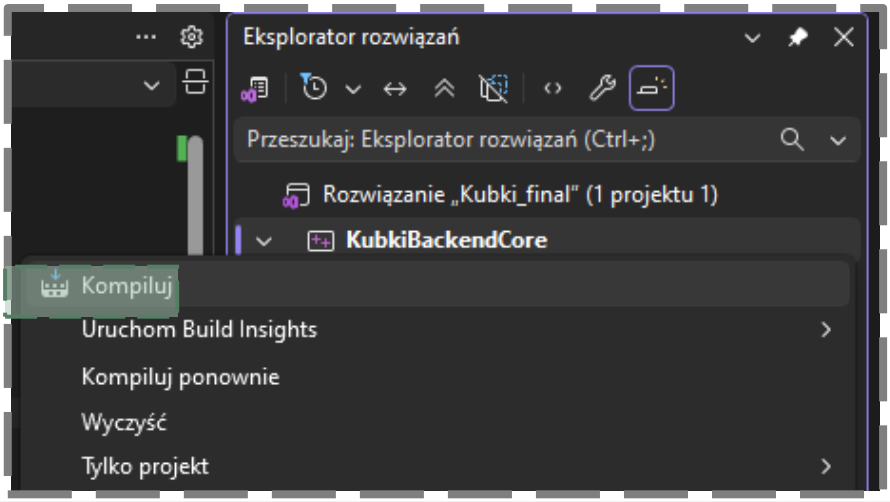
Zawartość drzewka projektu:



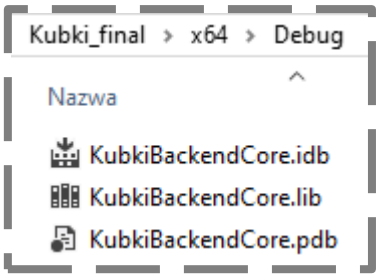
Do plików *TSubstance.cpp* oraz *TCup.cpp* należy dołączyć plik *pch.h*:



W tym momencie powinna być możliwość kompilacji biblioteki. Kliknij PPM na nazwę projektu w Eksploratorze rozwiązań i wybierz opcję ” Kompiluj”:

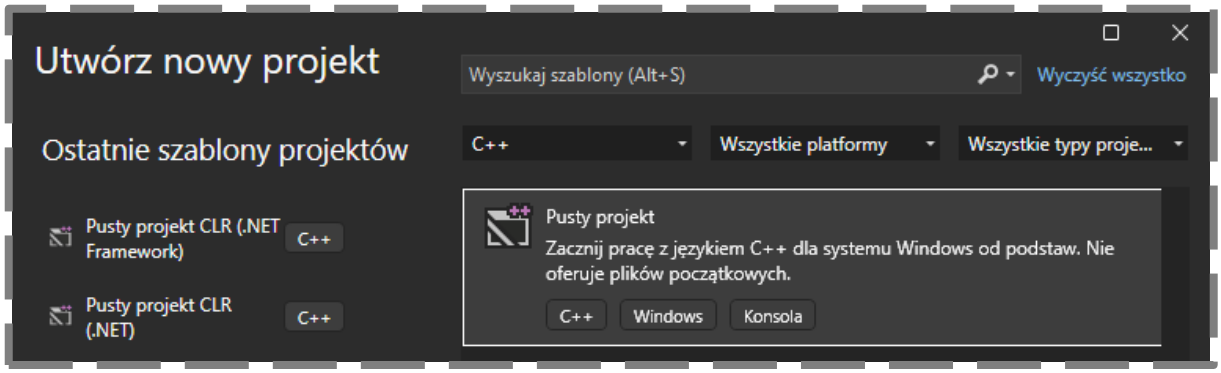


W efekcie w folderze projektu powinny zostać wygenerowane pliki:

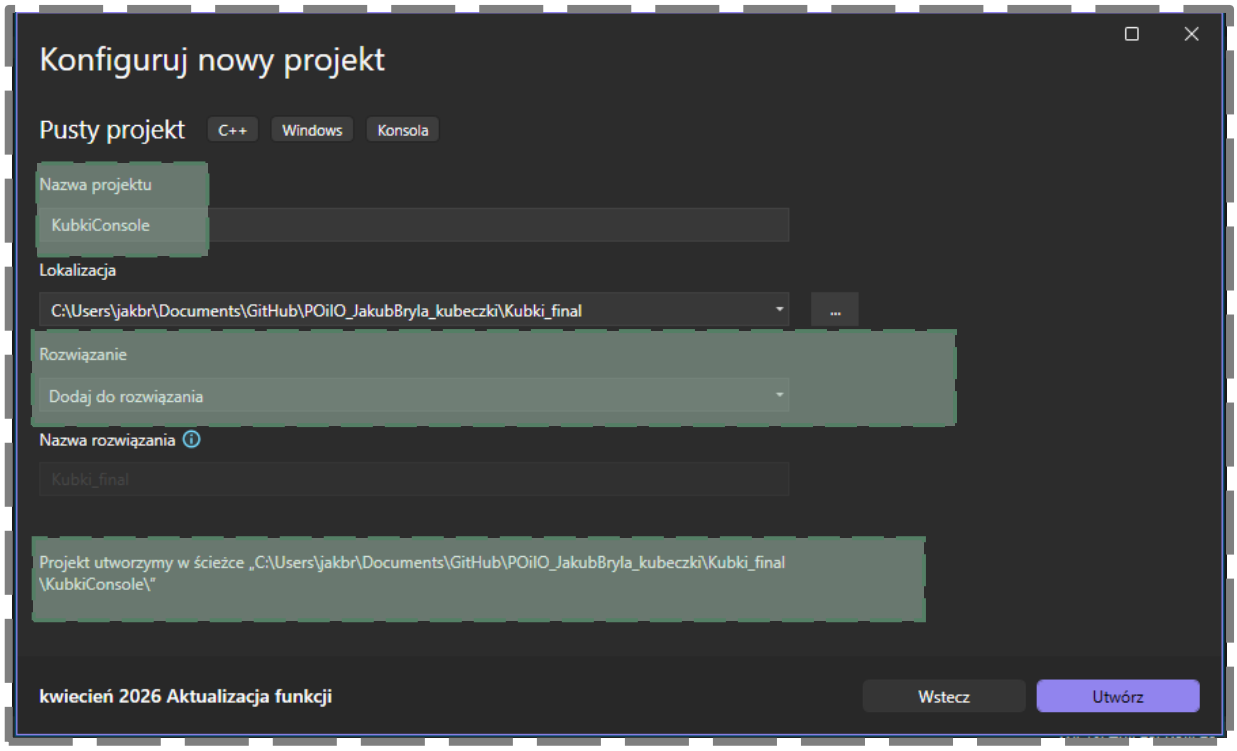


ZADANIE 2.2 – Utworzenie projektu aplikacji konsolowej

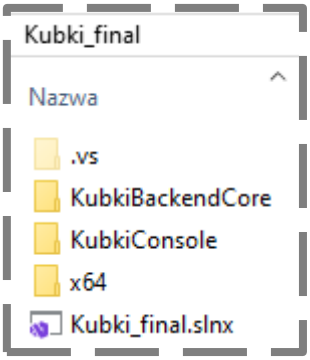
Utworzymy teraz projekt konsoli – **nie zamykaj projektu z zadania 2.1**. Dla nowego projektu wybierz szablon „Pusty projekt”:



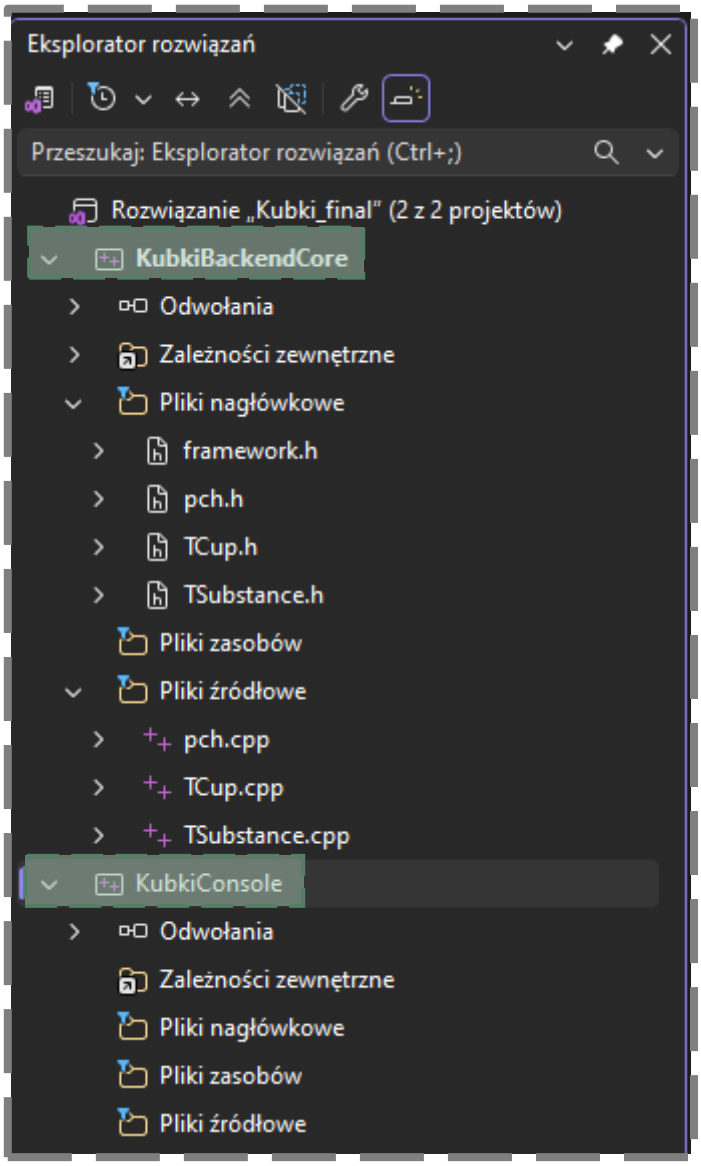
Następnie nadaj mu nazwę, która jednoznacznie będzie wskazywać, że jest to program konsoli. W „Rozwiązanie” wybierz opcję „Dodaj do rozwiązania”. Upewnij się, że lokalizacja projektu znajduje się w folderze, w którym zamieszczony jest projekt z zadania 2.1.



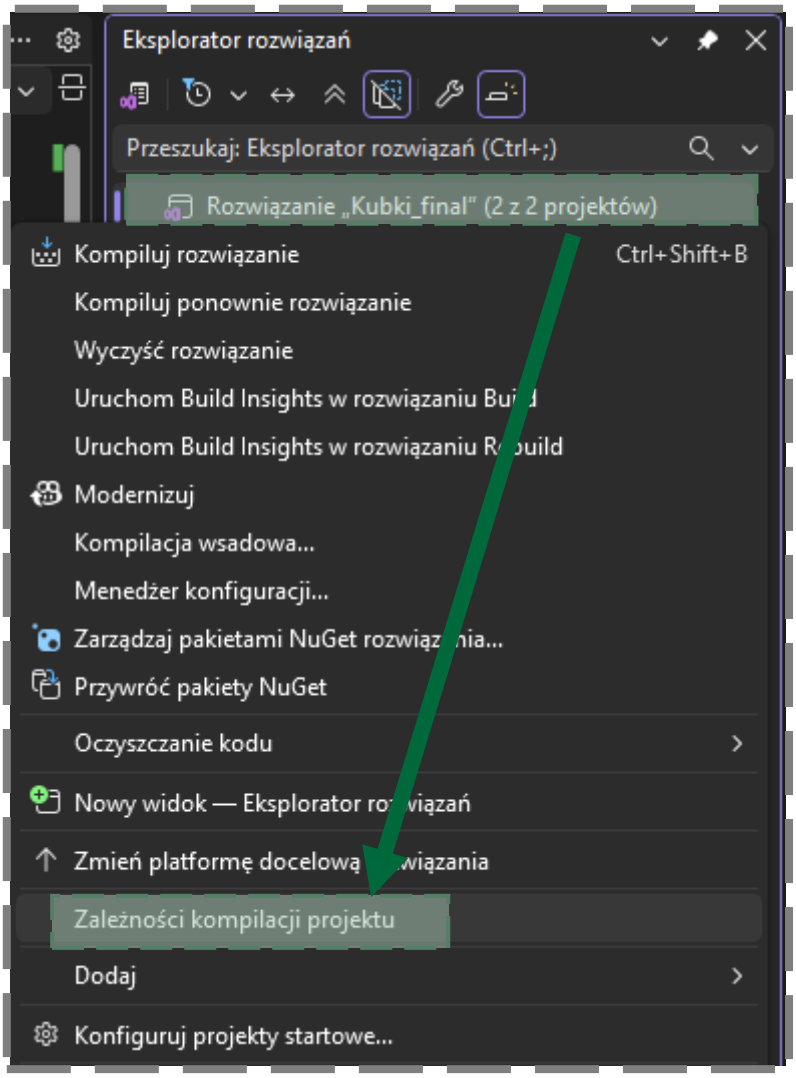
Po utworzeniu projektu, główny folder w repozytorium lokalnym powinien wyglądać następująco:



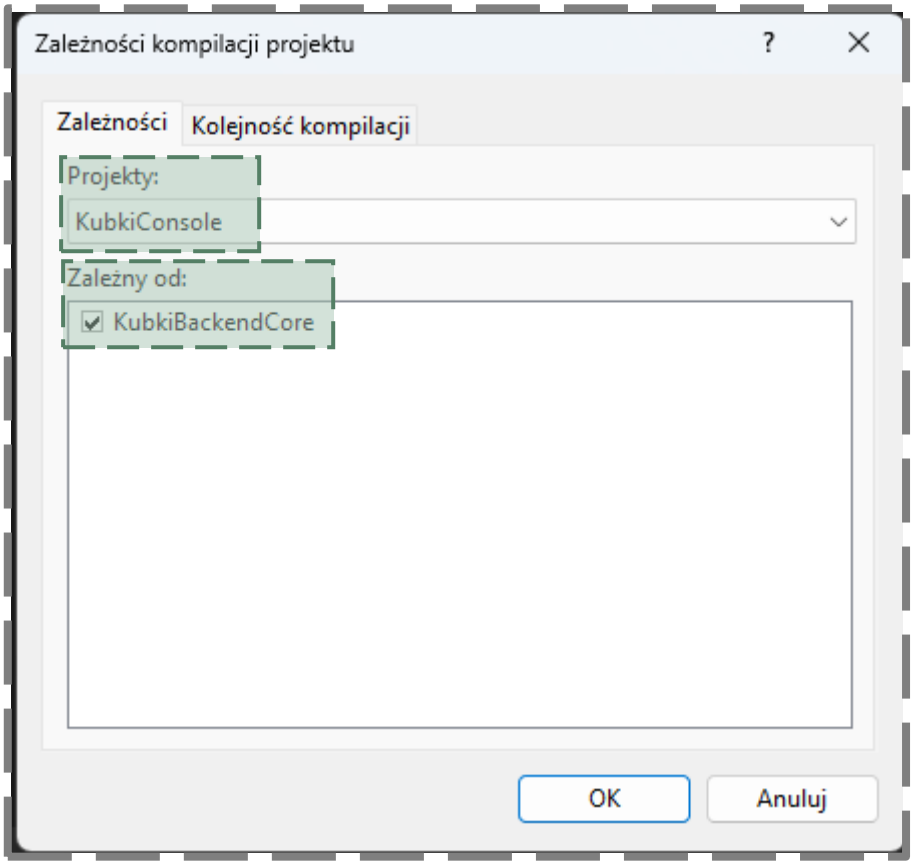
Natomiast drzewko Rozwiązania wygląda następująco – zawiera dwa projekty:



Od razu możemy określić zależności pomiędzy dwoma projektami znajdującymi się w ramach Rozwiązania VisualStudio. W tym celu kliknij PPM na Rozwiązanie i wybierz opcję „Zależność kompilacji projektu”:

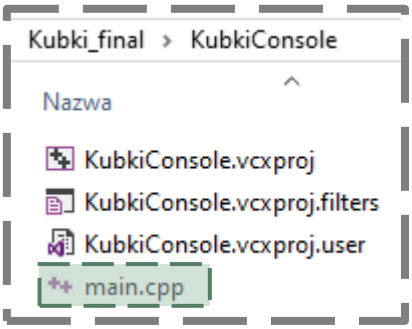


W nowo otwartym oknie wybierz w „Projekty” nowo utworzony projekt konsoli. Następnie zaznacz jego zależność od wcześniej utworzonej biblioteki statycznej. Niniejsze działanie określa kolejność kompilacji projektów w obrębie Rozwiązania – najpierw musimy przygotować (skompilować) bibliotekę, dopiero później projekt, który od niej zależy.

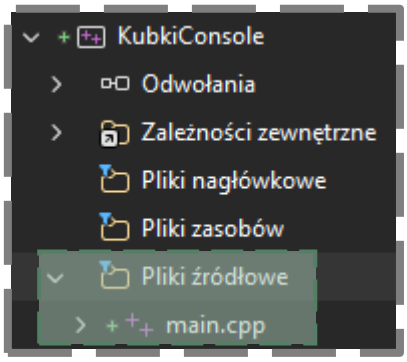


W celu zmiany kolejnych ustawień dla nowo utworzonego projektu musimy do niego dodać plik *.cpp* – dopiero wtedy pusty projekt zostanie powiązany z językiem C++ i w ustawieniach projektu pojawią się interesujące nas opcje.

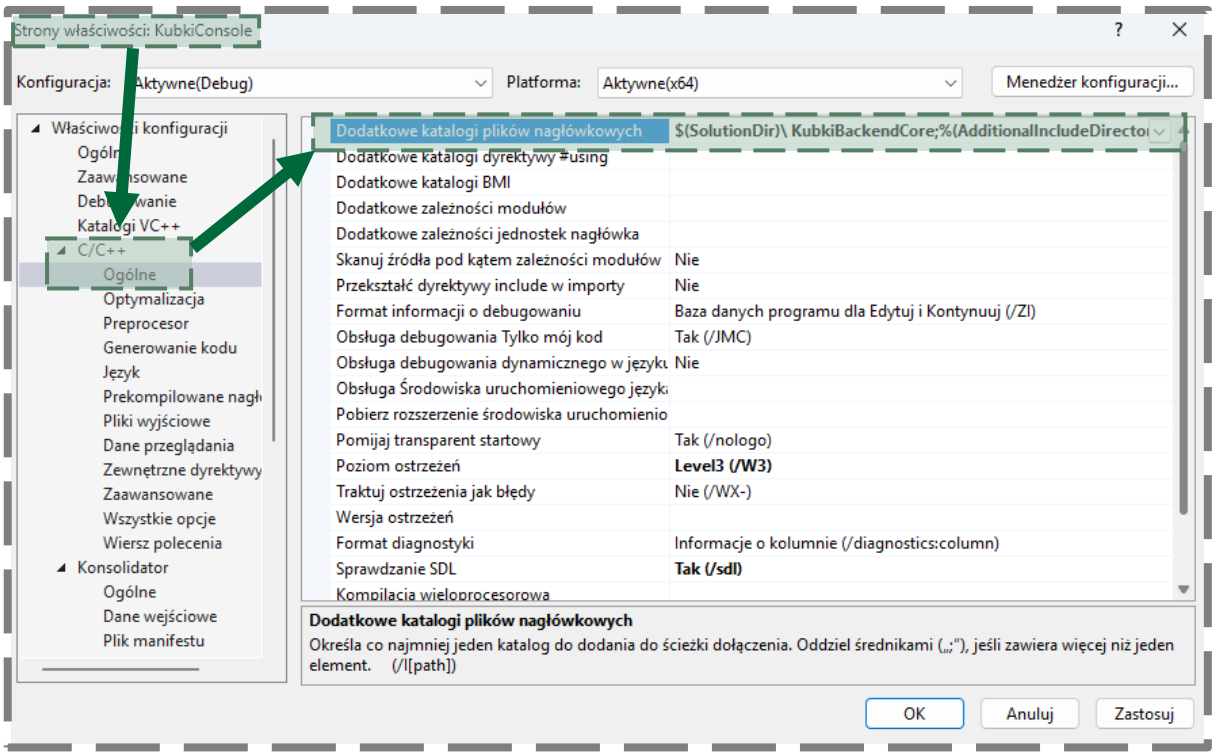
Skopiuj plik z funkcją *main* z jednej z wcześniejszych wersji programu konsoli:



Kliknij PPM na folder „Pliki źródłowe” w nowo utworzonym projekcie, wybierz opcję „Dodaj” → „Istniejący element” i wskaż skopiowany plik. Widok drzewka po powyższej akcji:



Przejdź do Właściwości projektu – kliknij PPM na nazwę projektu w Eksploratorze rozwiązań i wybierz ostatnią opcję z listy. W nowo otwartym oknie, w lewym menu, przejdź do zakładki „C/C++” → „Ogólne”. Z prawej strony okna wybierz opcję „Dodatkowe katalogi plików nagłówkowych” i kliknij „Edytuj”. Dodaj ścieżkę do projektu z zadania 2.1.

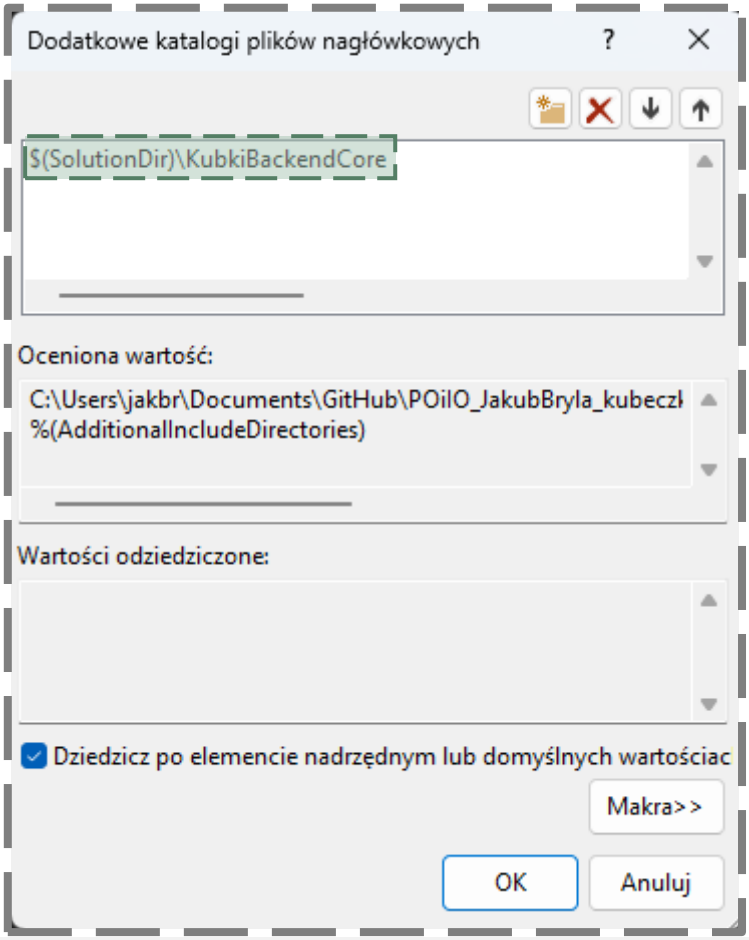


W celu zapewnienia poprawnego działania Rozwiązania VisualStudio, po skopiowaniu repozytorium, należy unikać ścieżek absolutnych przy podawaniu lokalizacji plików/bibliotek. Rozwiązaniem problemu jest stosowanie makr VisualStudio:

Makro	Co oznacza?
$(SolutionDir)	Katalog .sln
$(ProjectDir)	Katalog projektu
$(OutDir)	Katalog wyjściowy
$(Configuration)	Debug/Release
$(Platform)	x86/x64

W powyższym przykładzie należy podać ścieżkę lokalizacji dodatkowych plików nagłówkowych:

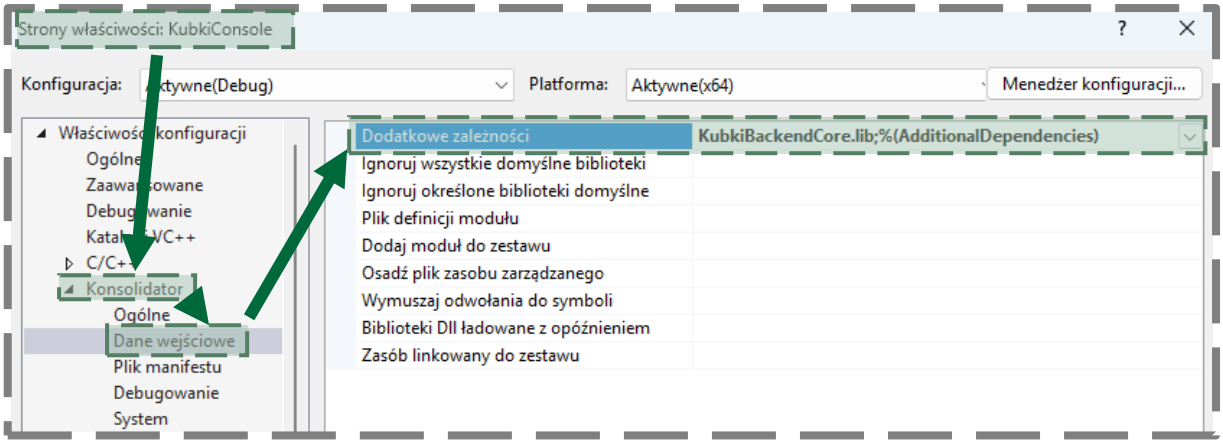
```
$(SolutionDir)KubkiBackendCore
```



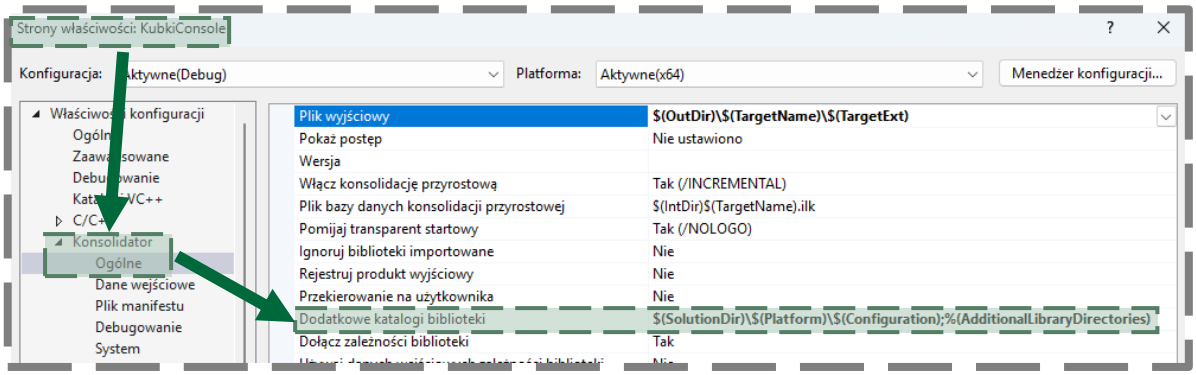
Powyższa zmiana ustawień pozwala znaleźć kompilatorowi pliki nagłówkowe biblioteki statycznej zaimplementowanej w zadaniu 2.1. Od teraz w naszym nowym projekcie możemy stosować bez komunikatu o błędzie – nieznanych plikach:

```
1  #include "TSubstance.h"
2  #include "TCup.h"
```

Wracamy do ustawień nowego projektu. W lewym menu wybierz opcję „Konsolidator” → „Dane wejściowe”. Z prawej strony wybierz „Dodatkowe zależności” i dodaj do nich nazwę skompilowanej biblioteki z zadania 2.1. W tym kroku konsolidator (linker) zostaje zapoznany z kodem funkcji bibliotecznych.



Pozostaje dodać informację o lokalizacji biblioteki. We Właściwościach nowego projektu, w lewym menu, wybierz opcję: „Konsolidator” → „Ogólne”. Z prawej strony okna wybierz opcję „Dodatkowe katalogi biblioteki” i wskaż lokalizację pliku **.lib* – skompilowanej biblioteki statycznej z zadania 2.1.



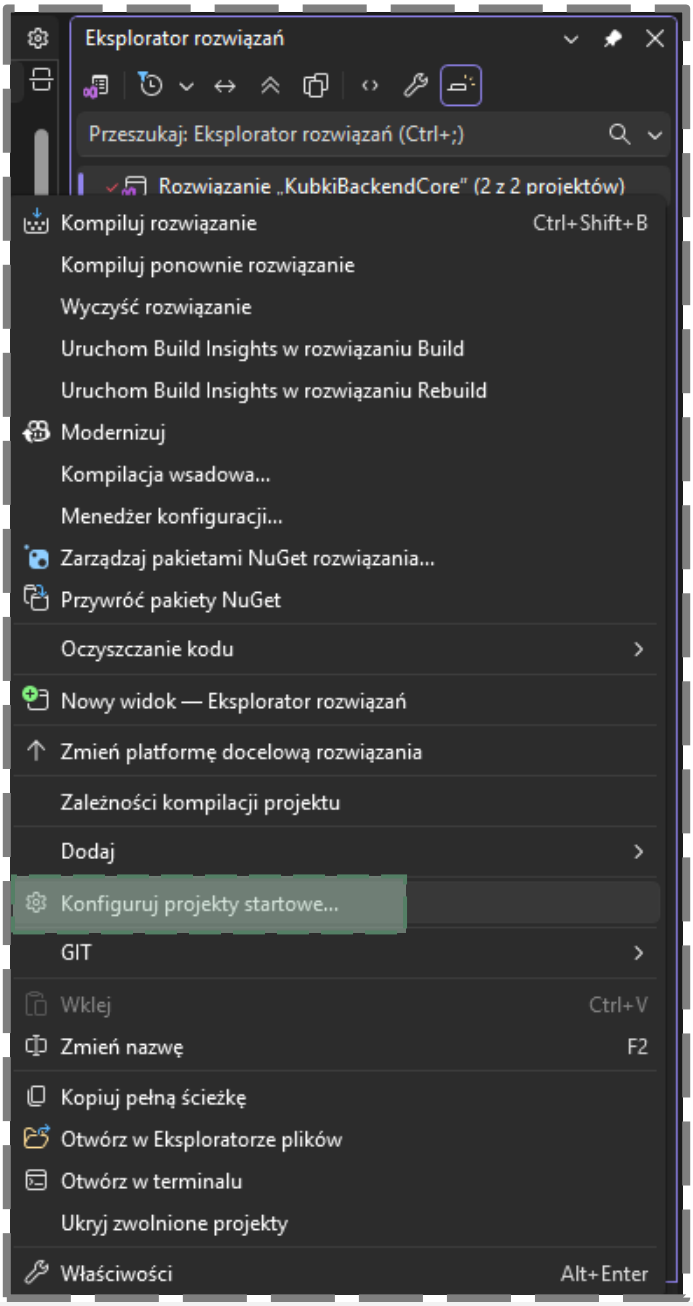
W powyższym przykładzie należy podać ścieżkę lokalizacji biblioteki statycznej korzystając z makr VisualStudio:

```
$(SolutionDir)x64\Debug
```

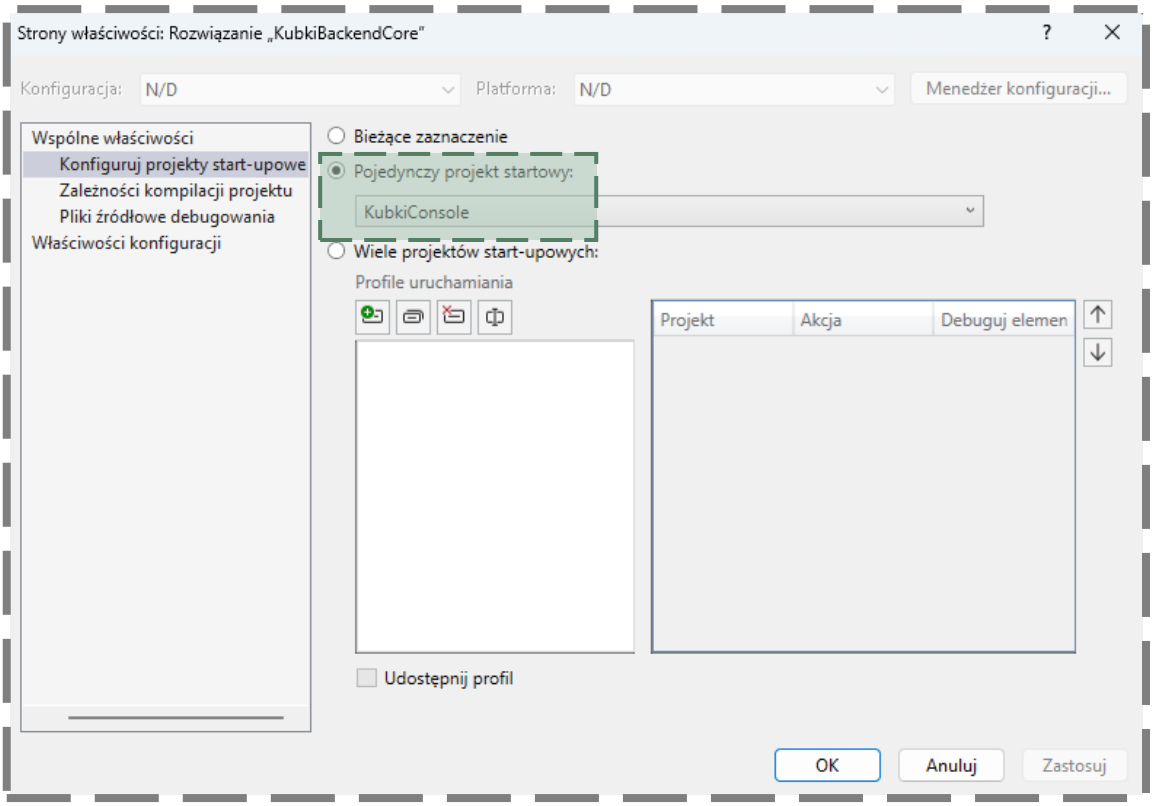
lub bardziej uniwersalnie:

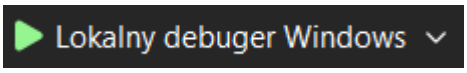
```
$(SolutionDir)$(Platform)\$(Configuration)
```

Wracając do opcji Rozwiązania należy określić projekt nadrzędny – projekt, który będzie uruchamiany po kompilacji (skrót klawiszowy *F5*). Kliknij PPM w Eksploratorze rozwiązań dla Rozwiązanie i wybierz opcję „Konfiguruj projekty startowe”.

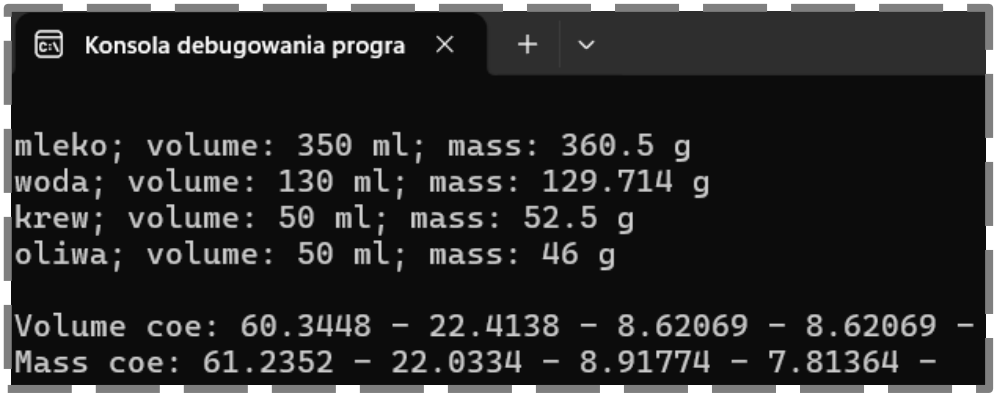


W nowo otwartym oknie w opcji „Pojedynczy projekt startowy:” wybierz projekt konsoli:



Uruchom kompilację dla Rozwiązania – przycisk 

lub skrót klawiszowy *F5*. W rezultacie powinieneś zobaczyć okno konsoli z wykonanym kodem z funkcji *main*:



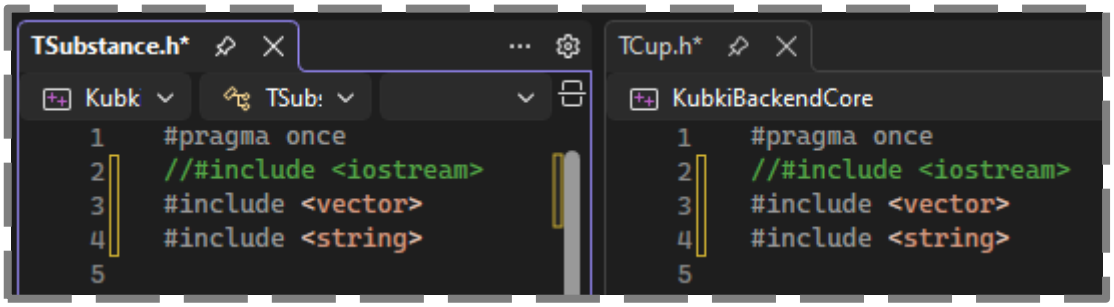
W tym momencie kod funkcjonalny projektu został umieszczony w bibliotece statycznej, możliwej do dołączenia i wykorzystania w innych projektach. Kod testowy, funkcja *main*, trafiła do osobnego projektu w ramach Rozwiązania VisualStudio.

ZADANIE 2.3 – Modyfikacja kodu klas, utworzenie wrappera konsoli

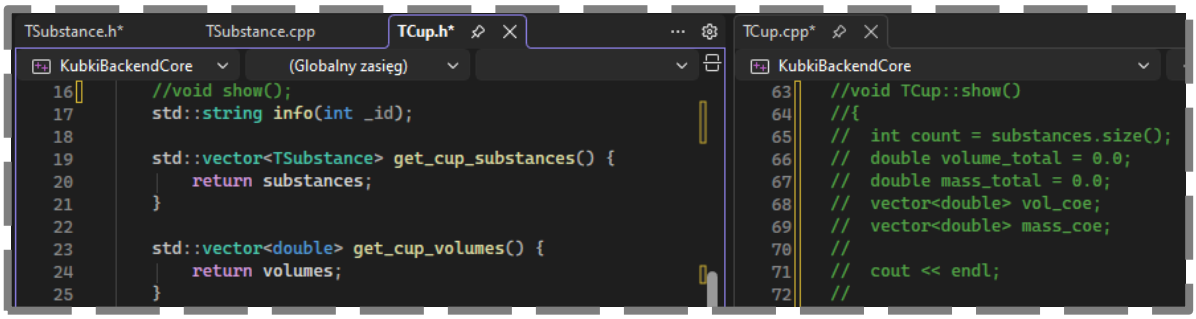
W celu wykorzystania zalet nowej architektury programu przejdziemy do uporządkowania obecnego kodu. Chcemy, aby wszystkie funkcje związane z konsolą trafiły do projektu *KubkiConsole*. Celem jest uniezależnienie kodu biblioteki statycznej od wyjścia, tj. sposobu wyświetlania informacji użytkownikowi.

Usunięcie w bibliotece zależności od biblioteki `<iostream>`

Wyświetlanie informacji w konsoli jest możliwe po załączeniu w pliku biblioteki `<iostream>`. Przejrzyjmy pliki biblioteki i zakomentujmy instrukcję `#include`:



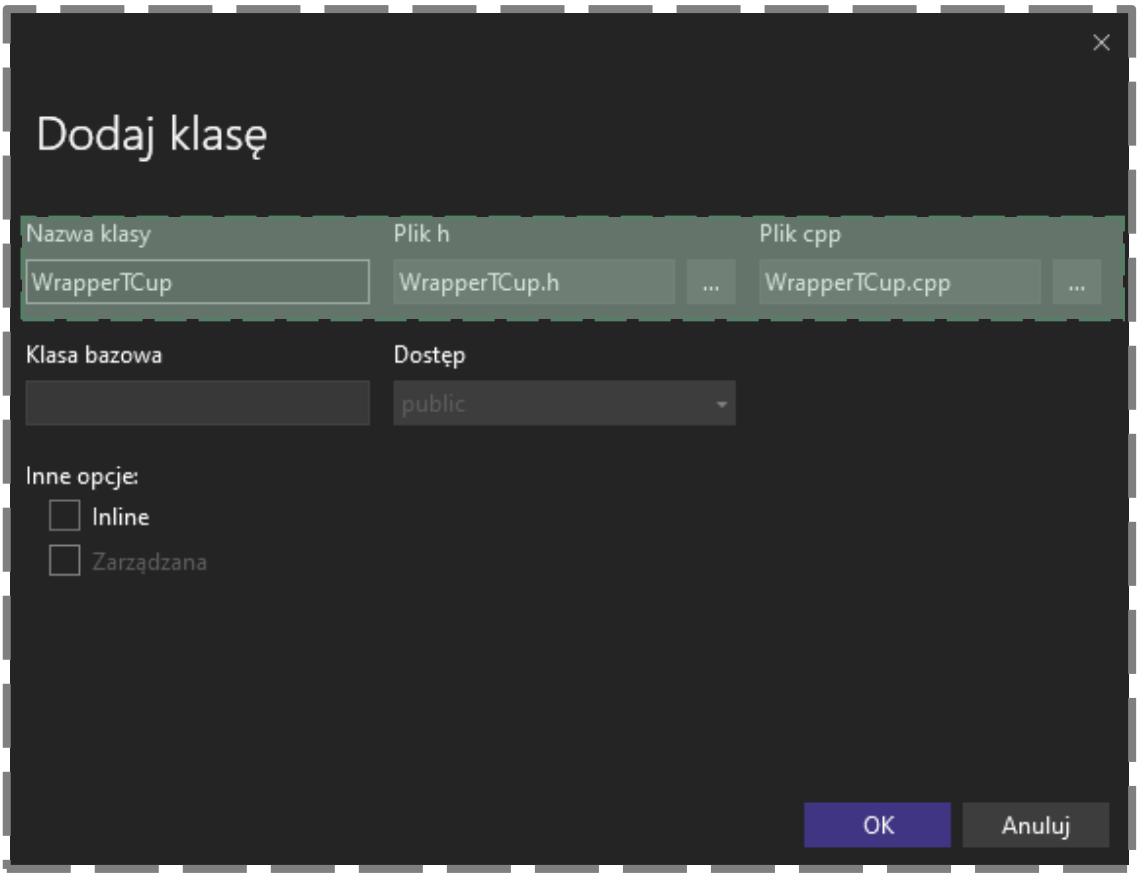
Przejrzyjmy kod plików źródłowych. W moim przypadku jedyny błąd występuje dla funkcji `show()` klasy `TCup` – występuje w niej wywołanie funkcji `cout`. Chwilowo zakomentuj cały blok funkcji `show()`. Tak samo, zakomentuj deklarację funkcji w pliku nagłówkowym:



Utworzenie Wrappera dla klasy `TCup`

Odwołania do biblioteki `<iostream>` występowały jedynie w klasie `TCup`. Utworzymy teraz specjalną klasę w projekcie `KubkiConsole`, która będzie nadbudowywać klasę `TCup`. Nowa klasa, o nazwie `WrapperTCup`, będzie przechowywać obiekt klasy `TCup` jako parametr. Dodatkowo, zostaną w niej zaimplementowane odpowiedniki funkcji klasy `TCup`, do których chcemy mieć dostęp z poziomu konsoli.

Zacznijmy od utworzenia nowej klasy w projekcie `KubkiConsole`:



Spójrzmy na plik nagłówkowy:

WrapperTCup.h

```
1  #pragma once
2  #include "TCup.h"
3
4  class WrapperTCup
5  {
6          TCup* cup = new TCup;
7
8  public:
9          ~WrapperTCup();
10
11         void add(TSubstance substance, double volume_in_ml);
12         void add(std::string name, double volume_in_ml);
13         void show();
14 };
15
```

Kwestią charakterystyczną dla wrapperów jest:

- przechowywanie innego obiektu w postaci wskaźnika,
- definicja destruktora usuwającego dynamicznie utworzony obiekt.

Dodatkowo, jak wspomniałem wcześniej, klasa posiada funkcje wywoływane w pliku `main` – linijka 11-13.

Spójrzmy na plik źródłowy:

WrapperTCup.cpp

```
1  #include "WrapperTCup.h"
2  #include <iostream>
3  #include <vector>
4
5  using namespace std;
6
7
8  WrapperTCup::~WrapperTCup()
9  {
10         delete cup;
11         cup = nullptr;
12 }
13
14 void WrapperTCup::add(TSubstance substance, double volume_in_ml)
15 {
16         cup->add(substance, volume_in_ml);
17 }
18
19 void WrapperTCup::add(std::string name, double volume_in_ml)
20 {
21         cup->add(name, volume_in_ml);
22 }
23
24 void WrapperTCup::show()
25 {
26         std::vector<TSubstance> substances = cup->get_cup_substances();
27         std::vector<double> volumes = cup->get_cup_volumes();
28
29          skopiowana treść funkcji show() z klasy TCup
30 }
31
```

Funkcje zdefiniowane wewnątrz pliku:

- destruktor: usuwa obiekt klasy `TCup` i zmienia wartość wskaźnika,
- funkcje `add`: wywołują funkcje `add` klasy TCup,
- funkcja show(): pobiera wektory `substances` i `volumes` z obiektu `cup`. Następnie wyświetla informacje o kubku zgodnie z tym jak było to robione wcześniej – kopiujemy kod z klasy `TCup`.

W tym momencie powinieneś móc skompilować Rozwiązanie.

Status dodania substancji do kubka

Sprawdźmy jeszcze jak moglibyśmy wyświetlić informację o błędzie lub sukcesie dodania substancji do kubka. Funkcje `add` klasy `TCup` nie zwracają wartości. Skorzystamy z tego i zmodyfikujemy je w taki sposób, aby zwracały liczbę odpowiadającą wynikowi działania funkcji. Umawiamy się na przypisanie znaczenia zwracanym wartością:

- 1 – nowa substancja została dodana do kubka,
- 2 – substancja znajdowała się w kubku, dolewamy,
- -1 – brak substancji o podanej nazwie w menu substancji.

Spójrzmy na zmiany w kodzie w klasie `TCup`.

TCup.h

```
int add(TSubstance substance, double volume_in_ml);
int add(std::string name, double volume_in_ml);

```

TCup.cpp

```
int TCup::add(TSubstance substance, double volume_in_ml)
{
        int status = 0;

        string new_name = substance.get_name();
        int substance_pos_in_cup = get_substance_id(&substances, new_name);

        if (substance_pos_in_cup == -1)
        {
                substances.push_back(substance);
                volumes.push_back(volume_in_ml / 1e6);
                status = 1;
        }
        else {
                this->volumes[substance_pos_in_cup] += volume_in_ml / 1e6;
                status = 2;
        }

        return status;

}

int TCup::add(std::string name, double volume_in_ml)
{
        int status = 0;

        int _id = get_substance_id(&substance_menu, name);

        if (_id >= 0)
        {
                status = this->add(substance_menu[_id], volume_in_ml);
        }
        else {
                status = -1;
        }

        return status;
}

```

Wartość zmiennej status wykorzystywana jest w klasie WrapperTCup do wyświetlenia odpowiednich informacji w konsoli:

WrapperTCup.h

```
1  #pragma once
2  #include "TCup.h"
3
4  class WrapperTCup
5  {
6          TCup* cup = new TCup;
7
8  public:
9          ~WrapperTCup();
10
11         void add(TSubstance substance, double volume_in_ml);
12         void add(std::string name, double volume_in_ml);
13         void show();
14
15 private:
16         void print_add_status(std::string name, int status, double volume);
17 };
18
```

WrapperTCup.cpp

```
1  void WrapperTCup::add(TSubstance substance, double volume_in_ml)
2  {
3          int status = cup->add(substance, volume_in_ml);
4
5          string name = substance.get_name();
6          this->print_add_status(name, status, volume_in_ml);
7  }
8
9  void WrapperTCup::add(string name, double volume_in_ml)
10 {
11         int status = cup->add(name, volume_in_ml);
12         this->print_add_status(name, status, volume_in_ml);
13 }
14
15 void WrapperTCup::print_add_status(string name, int status, double vol)
16 {
17         switch (status)
18         {
19         case -1:
20                 cout << "Nieznana substancja: " << name << endl;
21                 break;
22         case 1:
23                 cout << "Dodano nowa substancje do kubka \"" << name << "\"";
24                 cout << " w ilosci: " << vol << "ml.\n";
25                 break;
26         case 2:
27                 cout << "Dolano \"" << name << "\" do kubka";
28                 cout << " w ilosci: " << vol << "ml.\n";
29                 break;
30         default:
31                 cout << "Nieznany status funkcji add: " << status << endl;
32                 break;
33         }
34 }
35
```

Wynik uruchomienia programu:

```
Konsola debugowania programu Microsoft Visual Studio
Nieznana substancja: kwas
Dodano nowa substancje do kubka "mleko" w ilosci: 50ml.
Dodano nowa substancje do kubka "woda" w ilosci: 100ml.
Dodano nowa substancje do kubka "krew" w ilosci: 50ml.
Dodano nowa substancje do kubka "oliwa" w ilosci: 50ml.
Dolano "woda" do kubka w ilosci: 30ml.
Dolano "mleko" do kubka w ilosci: 300ml.

mleko; volume: 350 ml; mass: 360.5 g
woda; volume: 130 ml; mass: 129.714 g
krew; volume: 50 ml; mass: 52.5 g
oliwa; volume: 50 ml; mass: 46 g

Volume coe: 60.3448 - 22.4138 - 8.62069 - 8.62069 -
Mass coe: 61.2352 - 22.0334 - 8.91774 - 7.81364 -
```

3. Podsumowanie

Niniejsza instrukcja przedstawia bardzo ważne narzędzia z punktu widzenia inżynierii programowania.

Dzięki zastosowaniu bibliotek statycznych zyskujesz możliwość współdzielenia kodu między projektami. Pracując nad uniwersalnym kodem, np. sterowaniem silników krokowych, zyskujesz możliwość dołączenia biblioteki do wielu projektów, bez konieczności kopiowania plików i tworzenia wielu ich wersji. Dołączając bibliotekę do projektu należy:

- wskazać miejsce plików nagłówkowych,
- dodać nazwę biblioteki,
- wskazać lokalizację pliku *.lib* biblioteki.

Warto więc zadbać, aby pliki biblioteki znajdowały się w ogólnym folderze, niezwiązanym z żadną konkretną aplikacją.

Stosowanie wielu projektów w ramach Rozwiązania Visual Studio pozwala na lepszą organizację projektu. Korzystając z powyższego narzędzia tworzymy prawidłową architekturę aplikacji. Dla przykładu aplikacji kubków powinniśmy:

- stworzyć projekt, bibliotekę statyczną, dla kodu backendu,
- stworzyć projekt konsoli do debugowania kodu backendu,
- stworzyć bibliotekę dla graficznego interfejsu użytkownika.

Dwa pierwsze projekty powinniśmy implementować równolegle. Niemniej podział kodu na dwa projekty pozwala rozdzielić część logiczną programu od części testowej. Finalnie przechodzimy do utworzenia projektu w języku C++/CLI. Co daje nam powyższy podział:

- backend, biblioteka statyczna, jest niezależna od środowiska uruchomieniowego. Znajduje się w niej jedynie kod odpowiedzialny za warstwę logiczną programu,
- błędy występujące na etapie tworzenia GUI są odseparowane od backendu i kodu dla konsoli. Znacznie łatwiej znaleźć przyczynę błędu – analizujemy mniejszą ilość kodu. Wyłączenie projekt GUI z kompilacji skutkuje dalszym działaniem rozwiązania Visual Studio – w omawianym przypadku w dalszym ciągu uruchomią się poprawnie projekty dla backendu i konsoli.
- dalsze rozbudowywanie programu, np. przejście do innego środowiska GUI, zmiana języku implementacji GUI, nie stanowi problemu. Tworzymy nowy projekt w ramach rozwiązania VisualStudio i korzystamy z pełni działającej biblioteki statycznej.

Dla zapewniania kompatybilności kodu powinniśmy korzystać z wrapperów. Wrappery powinny nadbudowywać klasy backendu w celu zapewnienia poprawnej pracy z konkretnym projektem.

Dla przykładu:

- w C++ korzystamy z typu `std::string`,
- w C++/Cli korzystamy z typu `String^`.

W takim przypadku wrapper powinien być odpowiedzialny za przekształcenie zmiennej typu `std::string` z backendu na zmienną typu `String^` i przekazanie jej do właściwego projektu zaimplementowanego w języku C++/CLI. Kluczowe pliki projektów powinny być wolne od składni innych języków programowania:

- backend powinien zawierać jedynie kod w języku C++,
- GUI powinno zawierać jedynie kod w języku C++/CLI.

Wszelkie przekształcenia typu zmiennych między projektami powinny występować w wrapper'ach. Dla powyższego przykładu, wrapper przekształcający typ zmiennej z `std::string` na `String^` powinien znajdować się w projekcie GUI. W razie zmian kodu w backendzie, rzutujących na GUI, wiemy że należy dokonać zmian w kodzie wrapper'a GUI – po raz kolejny wiemy, gdzie należy dokonać modyfikacji, analizujemy mniejszą ilość kodu.